JWT 게시판 구현 보고서

01 서비스 소개와 구현 범위

학번 202212215 이름: 이재현 제출일: 2026.10.28

서비스 이름 음악 추천 게시판

기술 조합과 버전

- Vue 3.5.43 (Vite 8.3.4) / Express 5.3.0 (jsonwebtoken 9.0.3, sql.js 1.14.2) / Node.js 24.21.0, npm 11.19.0

GitHub 저장소 URL

- https://github.com/Jaehyeon3543/wsd-jwt

소스 코드 기준

- `main` 브랜치 / 코드 구현 기준 커밋 `55a0d93`. 최종 제출 기준 커밋 SHA는 LMS에 제출합니다.

서비스 소개와 선택 이유

좋아하는 음악을 다른 사람에게 추천하고, 다른 사람의 추천에서 새로운 음악을 찾고 싶은 사용자를 위한 게시판입니다. 누구나 로그인 없이 추천 글 목록과 내용을 읽을 수 있습니다. 로그인한 사용자는 어떤 음악을 왜 좋아하는지 글로 남길 수 있고, 자신이 쓴 글만 수정하거나 삭제할 수 있습니다. 읽기는 열려 있고 쓰기는 작성자에게만 허용되는 구조라서, 공개 조회와 JWT 인증·작성자 권한 검사를 함께 연습하기에 적합한 주제라고 판단했습니다.

Vue + Express를 선택한 이유는 수업에서 다룬 게시판 CRUD 코드와 같은 조합이어서 이미 수업에서 배운 내용 위에 인증과 권한 검사만 새로 붙이는 데 집중할 수 있기 때문입니다. 프론트와 백엔드가 모두 JavaScript라서 토큰이 브라우저에서 만들어진 요청 헤더부터 서버 검증까지 하나의 언어로 추적할 수 있다는 점도 생각했습니다. 제공 템플릿에서는 인증 TODO(BE-01~03, FE-01~04)을 구현했고, 화면 문구를 음악 추천 주제로 바꾸었습니다.

관련 파일에 파일과 함께 관련된 함수를 따로 괄호로 표시했습니다.

기능	완료 여부	관련 파일	비고 또는 제한
공개 목록과 상세	완료	`backend-express/src/app.js`, `frontend-vue/src/App.vue`	제공 코드 유지, 비로그인 200 확인
로그인과 JWT 발급	완료	`backend-express/src/auth.js` (`issueToken`), `frontend-vue/src/api.js` (`login`)	HS256, TTL 기본 90초
인증과 내 정보 조회	완료	`auth.js` (`currentUserId`), `app.js` (GET /api/me)	Alice·Bob 각각 200 확인
본인 글 작성·수정·삭제	완료	`app.js` (POST·PUT·DELETE), `auth.js` (`assertOwner`), `api.js` (`canEdit`)	작성자는 토큰에서 결정

기능	완료 여부	관련 파일	비고 또는 제한
만료·로그아웃·오류 표시	완료	`api.js` (`request`·`friendlyErrorMessage`·`clearSession`), `App.vue` (`run`·`logout`)	새로고침 시 로그인 해제 (의도된 설계)
DB 데이터 유지	완료	`backend-express/src/store.js` (제공 코드)	실제 서버 재시작 후 유지 확인 (T15)

웹서비스설계 02

구조와 변경 내역

브라우저 → API 서버 → SQLite의 연결 구조 그림. (AI에게 그려달라고 요청했습니다.)

```
[브라우저: Vue 빌드본]
   |  App.vue  →  api.js request()
  | 토큰 위치: api.js 모듈 변수 session (메모리)
   |
   |  fetch('/api/...')            ← 같은 출처로 요청
   ▼
[Vite preview 프록시]  /api → http://127.0.0.1:8000
   ▼
[Express API 서버 app.js]
  | 보호 API: currentUserId(req) → (글 존재 확인) → assertOwner → fields() 입력 검사
   ▼
[store.js → SQLite 파일 data/board.sqlite]
```

브라우저(Vue 빌드본)에서 요청을 보내면, 이 요청을 Vite preview 프록시에서 Express API 서버인 app.js로 요청을 보냅니다. app.js는 들어온 요청에 맞게 함수를 호출하고 필요하다면 SQLite(DB)에 접근해서 데이터를 가져옵니다. 이후 응답을 json 형태로 보내줍니다.

로그인 요청 흐름

로그인 폼 제출 → `api.js` `login()`이 `POST /api/auth/login`에 `{email, password}`를 보냅니다. 서버는 `fields()`로 입력 형식을 확인하고, `verifyCredentials()`로 비밀번호를 확인한 뒤 `issueToken()`으로 JWT를 만들어 `{accessToken, expiresIn, user}`를 돌려줍니다. 브라우저는 이 값을 `session` 변수에 저장합니다.

게시글 수정 요청 흐름

수정 저장 → `api.update()` → `request()`가 `authorizationHeaders()`로 `Authorization: Bearer TOKEN`을 붙여 `PUT /api/posts/{id}`를 보냅니다. 서버는 `currentUserId()`로 토큰을 검증해 사용자 id를 얻고, `postId()`·`getPost()`로 글의 존재를 확인한 뒤, `assertOwner()`로 작성자를 비교하고, 마지막에 `fields()`로 제목과 내용을 검사한 후 DB를 변경합니다.

변경한 파일과 함수	제공 코드의 원래 역할	내가 변경한 내용	변경 이유와 작성 출처
`backend-express/src/auth.js` 최상단 키 검사	`JWT_SECRET` 사용 TODO	빈 키·예시 키·64자리 hex가 아닌 키면 시작 시 Error, `Buffer.from(hex)`로 32바이트 키 생성	잘못된 키로 서버가 뜨는 것을 막기 위해 (출처: 본인 작성 (학습 중 Codex 예시 참고), AI 리뷰 후 수정)

변경한 파일과 함수	제공 코드의 원래 역할	내가 변경한 내용	변경 이유와 작성 출처
auth.js / issueToken	501 NOT_IMPLEMENTED	HS256, sub·iss·aud·exp 클레임으로 서명 및 토큰 발급	BE-01(출처: 본인 작성)
auth.js / currentUserId	데모 사용자 id 1 반환	Bearer 형식 확인, jwt.verify(HS256 고정, iss·aud·exp), exp·sub 형식, 실제 사용자 확인	BE-02 (출처: 본인 작성, AI 리뷰 후 수정)
auth.js / assertOwner	아무 검사도 하지 않음	토큰 사용자 id와 글 authorId비교, 틀리면 403 FORBIDDEN	BE-03 (출처: 본인 작성)
frontend-vue/src/api.js / login,	로그인 미구현 안내	응답의 accessToken·user를 메모리 변수에 저장	FE-1 (출처: 본인 작성, AI 리뷰 후 수정)
api.js / authorizationHeaders	빈 객체 {} 반환	토큰이 있을 때만 Authorization: Bearer 헤더 붙여준다.	FE-02 (출처: 본인 작성)
api.js / request, friendlyErrorMessage	서버 메시지 그대로 표시	401이면 clearSession(), 상태·code별 안내 문구, error.status·error.code 전달	FE-03 (출처: 본인 작성, AI 리뷰 후 수정)
api.js / canEdit	항상 true	로그인 사용자 id와 post.authorId 비교	FE-04 (출처: 본인 작성, AI 리뷰 후 수정)
frontend-vue/src/App.vue / run, logout, 템플릿	데모 배너, 로그인 상태 미반영	run() 종료 시 user를 세션과 동기화, 404면 상세 닫고 목록 갱신, 로그아웃·등록 버튼 비활성화, 비로그인 안내, 음악 추천 문구	FE-03·FE-04, 주제 반영 (출처: 본인 작성)
backend-express/src/app.js / fields 14행	!(k in limits)로 허용 필드 검사	!Object.hasOwn(limits, k)	in은 프로토타입 키(toString, constructor)도 허용해 추가 필드가 201로 통과했음 (출처: AI 수정, 본인 요청·확인)
app.js / 오류 처리기 99~100행	JSON 파싱 실패만 400	body-parser의 클라이언트 오류(error.expose) 전체를 400	64KB 초과 본문이 500 INTERNAL_ERROR로 응답되었음 (출처: AI 수정, 본인 요청·확인)
frontend-vue/src/style.css / button:disabled	cursor: wait	cursor: not-allowed	비활성 버튼이 로딩 중처럼 보였음 (출처: 본인 수정)

store.js, users.js, errors.js, 라우트 구조, vite.config.js, tools/check_api.py는 제공 코드를 그대로 사용했습니다.

!(k in limits)로 허용 필드 검사를 수정한 것은 AI의 검사 도중 발견한 점으로. AI의 수정의 제안을 읽어보고 검색해 찾아보고 난 후 수정했습니다.

로컬 실행 기록

운영체제와 실행 환경

- Windows + WSL2 Ubuntu 26.04.1 / Node.js (24.21.0), npm (11.19.0), Python (3.14.4) / 백엔드 `npm start` (backend-express), 프론트 `npm run build` → `npm run preview` (frontend-vue)

SQLite 파일 경로

- `backend-express/data/board.sqlite` (`.env`의 `DB_FILE`)입니다. 서버 시작 시 폴더, DB 파일, `posts` 테이블이 없으면 자동으로 만듭니다.

- 프론트 빌드본(`npm run build` → `npm run preview`)을 http://127.0.0.1:4173 에서 열고, `/api` 프록시로 8000번 백엔드와 연결된 상태에서 T01~T17 화면 검사를 수행했습니다. 모든 화면 증거의 주소창이 `127.0.0.1:4173`입니다.
- GitHub에 push한 저장소를 새 폴더 `~/wsd-01/wsd-jwt-check`에 clone하고 다음 순서로 실행했습니다. 실제 `.env`와 DB 파일은 저장소에 없으므로, clone한 폴더에서 처음부터 만들었습니다.
  1. 백엔드 `npm ci` → `cp .env.example .env` → 예시 키 그대로 `npm start`: `Error: JWT_SECRET` 은 새로 생성한 key여야 합니다. 예시키를 그대로 사용하면 시작이 거부됩니다. (T00_02_clone_refuse)
  2. 키 생성 후 직접 .env 파일에 JWT_SECRET에 넣습니다. → `npm start`: 게시판 시작 코드 `http://127.0.0.1:8000/` (T00_03_clone_backend -> 첫 시도에 서버가 켜져있어서 껏다가 다시 켰습니다.).
  3. 프론트 `npm ci` → `npm run build` → `npm run preview`: `Local:http://127.0.0.1:4173/` (T00_04_clone_build -> 첫 시도에서 프론트가 켜져있어서 껏다가 다시 켰습니다.).
  4. 4173번 화면에서 "게시판이 준비되었습니다."와 **게시글 목록 0개**가 표시되었습니다 (T00_05_clone_screen). 목록 API가 프록시를 거쳐 clone한 백엔드에 연결되었고, DB 파일 없이 시작해 빈 DB가 새로 만들어졌다는 뜻입니다. 기존 작업 폴더의 DB에는 글이 5개 있었습니다. (SQLite의 파일 경로는 위에 작성되어있습니다.)
- clone 폴더에서 테스트 계정으로 로그인되는 것도 확인했습니다.

화면 캡처는 4장 "자동 검사와 로컬 배포 증거"에 있습니다. 자세한 재현 방법은 저장소 README의 "설치와 실행"에 정리했습니다.

핵심 코드 설명

줄 번호는 커밋 55a0d93 기준입니다. 발췌 코드는 읽기 쉽게 주석을 빼고 일부 줄을 합쳤으며, 동작은 원본과 같습니다.

3. 1 로그인과 JWT 발급

파일 / 함수

- backend-express/src/app.js -> POST /api/auth/login (45~60행)
- backend-express/src/auth.js -> issueToken (20~28행)
- backend-express/src/app.js -> fields (9~28행)

코드 발췌

```
// app.js
function fields(body, limits) {
  if (!body || Array.isArray(body) || typeof body !== "object" ||
      Object.keys(body).some((k) => !Object.hasOwn(limits, k)))
    throw new ApiError(400, "BAD_INPUT", "허용된 JSON 필드만 보내세요.");
  const result = {};
  for (const [k, max] of Object.entries(limits)) {
    if (typeof body[k] !== "string" || !body[k].trim() || [...body[k].trim()].length > max)
      throw new ApiError(400, "BAD_INPUT", `${k} 입력을 확인하세요.`);
    result[k] = k === "password" ? body[k] : body[k].trim();
  }
  return result;
}


// app.js
app.post("/api/auth/login", (req, res) => {
  const input = fields(req.body, { email: 254, password: 128 });
  const user = verifyCredentials(input.email, input.password);
  if (!user)
    throw new ApiError(401, "INVALID_CREDENTIALS", "이메일 또는 비밀번호를 확인하세요.");
  const accessToken = issueToken(user);
  res.json({ accessToken, expiresIn: Number(process.env.TOKEN_TTL || 90), user });
});

// auth.js
export function issueToken(user) {
  return jwt.sign({}, jwtSecret, {
    algorithm: "HS256",
    subject: user.id.toString(),
    issuer: "webservice-class",
    audience: "board-api",
    expiresIn: Number(process.env.TOKEN_TTL || 90),
  });
}
```

동작과 응답 코드 설명

- **입력 :** 요청 본문 {email, password}.

- **처리**

- 입력 검증 : `fields()`가 두 필드만 있는지, 문자열인지, 비어 있지 않은지, 길이 제한(254·128자) 안인지 검사합니다. 다른 필드가 있거나 형식이 틀리면 400 BAD_INPUT 오류를 던집니다.

- 계정 확인 : 다음으로 제공 함수 `verifyCredentials()`가 scrypt 해시를 `timingSafeEqual`로 비교합니다. 이를 통해 없는 계정이나 틀린 비밀번호는 모두 `null`이 되어 401 INVALID_CREDENTIALS로 응답합니다. 두 경우의 응답이 같아서, 응답만 보고 어떤 이메일이 가입되어 있는지 알아낼 수 없습니다.

- **토큰 발급 :** 계정 확인이 끝난 사용자에게만 `issueToken()`이 실행됩니다. 페이로드에는 비밀번호나 이름을 넣지 않고 밑에의 클레임만 담습니다. 서명은 `.env`의 64자리 hex를 32바이트로 바꾼 `jwtSecret`과 HS256으로 했습니다.
  - `sub`: 문자열 사용자 id입니다. JWT 표준이 문자열을 요구하므로 `toString()`으로 바꿔줍니다.
  - `iss=webservice-class`, `aud=board-api`(`.env`에 있는거 그대로)
  - `exp`: `expiresIn`이 UTC Unix 초로 계산합니다. 기본 90초이고, `TOKEN_TTL` 환경변수로 바꿀 수 있습니다.
- **응답 : 200** `{accessToken, expiresIn, user:{id, name, email}}`. `user`는 `findPublic()`이 만든 공개 정보입니다.
- **서버 시작 시 검사 :** `auth.js` 5~15행은 함수 밖 최상단에서 키를 검사합니다.이때 키가 없거나, 예시 문자열 그대로이거나, 64자리 hex가 아니면 모듈을 읽는 순간 Error가 나서 서버가 아예 시작되지 않습니다. 함수 안에서 검사하면 첫 로그인 요청이 와야 문제가 드러나고, 그 전까지 서버는 정상처럼 보입니다. 새 폴더에 clone해서 예시 키 그대로 `npm start`하면 바로 종료되는 것을 확인했습니다 (T00_02).

3. 2 브라우저 토큰 저장과 전송

파일 / 함수

- `frontend-vue/src/api.js`

-> `session`·`clearSession` (2~7행), `authorizationHeaders` (9~12행), `request` (42~62행), `login` (63~70행)

코드 발췌

```
let session = { accessToken: "", user: null };
export function clearSession() { session = { accessToken: "", user: null }; }

// authorizationHeaders
function authorizationHeaders() {
  if (!session.accessToken) return {};
  return { Authorization: `Bearer ${session.accessToken}` };
}

//friendlyErrorMessage
function friendlyErrorMessage(status, data) {
  if(status === 400) {
    return `잘못된 입력입니다. 확인해주세요.(${data.message})`;
  }
  if(status === 401) {
    if (data.code === "INVALID_CREDENTIALS") {
    return "이메일 또는 비밀번호가 잘못되었습니다.";
  }
  if (data.code === "TOKEN_REQUIRED") {
    return "로그인이 필요합니다.";
  }
```

```
  if (data.code === "INVALID_TOKEN") {
    return "로그인 정보가 올바르지 않습니다. 다시 로그인해주세요.";
  }
  if (data.code === "TOKEN_EXPIRED") {
    return "로그인 시간이 만료되었습니다. 다시 로그인해주세요.";
  }
  return "로그인 상태가 아닙니다. 다시 로그인해주세요.";   // 그 외 401
  }
  if(status === 403) {
    return "권한이 없습니다.";
  }
  if(status === 404) {
    return "요청한 게시글을 찾을 수 없습니다.";
  }
  return data.message || data.code || `HTTP ${status}`;
}

// request
export async function request(path, options = {}) {
  const response = await fetch(`/api${path}`, {
    ...options,
    headers: { "Content-Type":
"application/json", ...authorizationHeaders(), ...options.headers },
  });
  const data = response.status === 204 ? null : await response.json();
  if (!response.ok) {
    if (response.status === 401) clearSession();
    const error = new Error(friendlyErrorMessage(response.status, data));
    error.status = response.status; error.code = data.code;
    throw error;
  }
  return data;
}

// login
export async function login(email, password) {
  const data = await request("/auth/login", { method: "POST", body: JSON.stringify({ email,
password }) });
  session = { accessToken: data.accessToken, user: data.user };
  return data.user;
}
```

동작과 응답 코드 설명

- **로그인 응답 처리 :** `request()`가 실패하면 throw하니까 `login()`의 `session = ...` 줄은 성공(200)일 때만 실행됩니다. 실패한 로그인이 저장되거나 성공 메시지로 표시되지 않도록 했습니다. `App.vue`의 `run()`이 오류를 잡아 메시지만 보여 주도록 했습니다.
- **저장 위치 :** 토큰은 `session`에만 있고, `localStorage·sessionStorage`에는 저장하지 않았습니다. 메모리 변수는 탭을 닫거나 새로고침하면 사라지므로 토큰이 브라우저 저장소에 계속 남지 않습니다. `session`은 Vue의 `ref`가 아니라서 바뀌어도 화면이 자동으로 갱신되지 않습니다. 그래서 `App.vue`의 `run()`이 `finally`에서 `user.value = getSession().user`로 화면 상태를 맞춥니다.
- **Authorization 헤더 :** 이 헤더를 검사하는 함수는 request( ) 안에서 호출해서 토큰을 검사합니다. `authorizationHeaders()`는 토큰이 없으면 빈 객체를 돌려줍니다. 그래서 `Bearer undefined`나 `Bearer(공백)` 같은 잘못된 헤더를 보내지 않습니다. 로그아웃 후에는 요청(/me, 등록 등등..)에 Authorization 헤더가 들어있지 않음을 확인할 수 있습니다.

- **request ( ) 에서 성공 처리 :** 204를 제외한 성공은 그대로 return해줍니다. 삭제 성공 204는 본문이 없으므로 `response.json()`을 호출하지 않습니다. 호출하면 파싱 오류가 나서, 성공한 삭제가 실패처럼 보이게 됩니다.
- **실패 처리 :** 401을 받으면 `clearSession()`으로 즉시 로그인 상태를 해제합니다. 그 외의 실패는 `friendlyErrorMessage()`가 `data.code`에 따라 안내 문구를 고릅니다.
- **로그아웃 :** `App.vue` `logout()`이 `clearSession()`으로 토큰을 지우고 `user.value = null`, 입력을 초기화 합니다. 로그아웃 버튼은 로그인 상태가 아니면 `:disabled="!user"`로 비활성화됩니다.

상태와 code	안내 문구
401 `INVALID_CREDENTIALS`	이메일 또는 비밀번호가 잘못되었습니다.
401 `TOKEN_REQUIRED`	로그인이 필요합니다.
401 `INVALID_TOKEN`	로그인 정보가 올바르지 않습니다. 다시 로그인해주세요.
401 `TOKEN_EXPIRED`	로그인 시간이 만료되었습니다. 다시 로그인해주세요.
400	잘못된 입력입니다. 확인해주세요. (뒤에 서버 메시지)
403	권한이 없습니다. (로그인 상태는 유지)
404	요청한 게시글을 찾을 수 없습니다. (`App.vue`가 상세를 닫고 목록을 다시 불러옴)

서버 인증과 작성자 권한

3. 3 서버 JWT 검증

파일 / 함수

- `backend-express/src/auth.js` -> `currentUserId` (30~74행)

코드 발췌

```
//currentUserId
Export function currentUserId(req){
Const authheader = req.headers.authorization;
const match = /^Bearer (\S+)$/.exec(req.headers.authorization ?? "");
if (!match) throw new ApiError(401, "TOKEN_REQUIRED", "유효한 토큰이 필요합니다.");
let payload;
try {
  payload = jwt.verify(match[1], jwtSecret, {
    algorithms: ["HS256"], issuer: "webservice-class", audience: "board-api",
  });
} catch (error) {
  if (error instanceof jwt.TokenExpiredError)
    throw new ApiError(401, "TOKEN_EXPIRED", "토큰이 만료되었습니다.");
  throw new ApiError(401, "INVALID_TOKEN", "유효하지 않은 토큰입니다.");
}
if (typeof payload.exp !== "number" || typeof payload.sub !== "string" || !/^[1-
9]\d*$/.test(payload.sub))
  throw new ApiError(401, "INVALID_TOKEN", "유효하지 않은 토큰입니다.");
const user = findPublic(Number(payload.sub));
if (!user) throw new ApiError(401, "INVALID_TOKEN", "사용자를 확인할 수 없습니다.");
return user.id;
}
```

동작과 응답 코드 설명

- **입력 :** 요청의 `Authorization` 헤더.
- **형식 검사 :** 정규식으로 `Bearer <공백 없는 문자열>` 형태만 통과시킵니다. 다음 경우는 모두 401 TOKEN_REQUIRED입니다. (match 변수에 넣을 때)
  - 헤더가 없음
  - `Basic ...` 같은 다른 방식
  - `Bearer` 뒤가 비어 있음
- **서명 검증 :** `jwt.verify`를 통해서 검사합니다. `jwt.verify`는 토큰의 헤더, 페이로드와 서버 키로 HMAC-SHA256을 다시 계산해서 토큰에 붙은 서명과 비교합니다. 이때 서명은 토큰에서 꺼내 믿는 값이 아니라, 서버가 직접 계산한 값과 일치해야 하는 값입니다.
  - 페이로드의 `sub`를 2로 바꾸면 계산 결과가 달라져서 실패합니다. (서버 키로 다시 계산했을 때의 값과 토큰의 값이 다르므로 실패)
  - 서명 한 글자만 바꿔도 실패합니다.

- **알고리즘 고정 :** `algorithms: ["HS256"]`이 다음 토큰을 거부합니다.
  - `alg: none` 토큰 (서명 없음)
  - HS512처럼 다른 알고리즘으로 서명한 토큰
- **클레임 검사 :** `issuer·audience` 옵션으로 `iss·aud`의 존재와 값을 검사하고, `exp`가 지났는지도 확인합니다. `jwt.verify`가 검사하지 않는 것은 직접 확인했습니다.
  - `exp`가 아예 없는 토큰: 라이브러리는 통과시키지만, 영원히 유효한 토큰이 되므로 거부합니다.
  - `sub`가 `"1"`처럼 0으로 시작하지 않는 양의 정수 문자열인지 확인합니다. `"01"`, `"abc"`, 숫자 `1`은 거부합니다.
- **사용자 확인 :** 서명이 맞아도 그 사용자가 실제로 있는지 `findPublic()`으로 확인합니다. 예를 들어 `sub=999`는 거부합니다. 확인이 끝난 계정의 id만 라우트에 넘깁니다.
- **성공, 실패 응답 (실패 HTTP와 JSON 코드)**
  - 검증 성공 시에는 `currentUserId`는 응답을 보내지 않고 사용자 id(숫자)를 반환합니다
- 그 id를 받은 라우트가 성공 응답을 보냅니다.(`GET /api/me`는 **200)**, (`POST /api/posts` 은 **201** (+ `Location: /api/posts/{id}`)),(`PUT /api/posts/{id}`는 **200)**, (`DELETE /api/posts/{id}`는 **204** (본문 없음)) 입니다.
- 성공 응답에는 JSON `code`가 없습니다 (**해당 없음**). `code`는 오류 응답에만 포함됩니다
  - 실패에 대해서는 아래와 같습니다.
  - 만료(`TokenExpiredError`)는 401 TOKEN_EXPIRED
  - 나머지(변조, 다른 키, 잘못된 클레임, 없는 사용자)는 모두 401 INVALID_TOKEN
  - `app.js` 102행의 오류 처리기가 모든 401에 `WWW-Authenticate: Bearer` 헤더를 붙입니다.
- **확인 방법 :** 이에 대해서는 직접 해본 다음 스크린샷을 찍어 evidence에 넣어뒀습니다. 대표적으로 확인 가능한 오류와 스크린샷은 아래와 같습니다.
  - 토큰 없이 보낸 요청은 401 TOKEN_REQUIRED로 거부됩니다. (`T04_02_create_no_token`, `T11_01_logout_me`).
- 서명 첫 글자를 바꾼 토큰으로 `/me·POST·PUT·DELETE`를 보냈을 때 모두 401 INVALID_TOKEN이 나옵니다. (`T09_02_tamper_result1`, `T09_03_tamper_result2`).
- `TOKEN_TTL=3`으로 만료시킨 토큰은 401 TOKEN_EXPIRED로 거부됩니다. (`T10_02_me_expired`, `T10_03_modify_expired`).
- 정상 토큰은 검증을 통과해 `/me`가 200을 반환합니다 (`T05_02_me`).

3. 4 작성자 권한 검사

파일 / 함수

`- backend-express/src/auth.js`

-> `assertOwner` (76~80행), `backend-express/src/app.js` POST · PUT · DELETE (69~91행), `fields` (9~28행)

코드 발췌

```
// auth.js
export function assertOwner(userId, authorId) {
  if (userId !== authorId) throw new ApiError(403, "FORBIDDEN", "작성자만 수행할 수 있습니다.");
}

// app.js
```

```
app.post("/api/posts", (req, res) => {
  const userId = currentUserId(req);
  const p = store.create(fields(req.body, { title: 100, content: 5000 }), userId);
  res.status(201).location(`/api/posts/${p.id}`).json(p);
});
app.put("/api/posts/:id", (req, res) => {
  const userId = currentUserId(req), id = postId(req), p = getPost(id);
  assertOwner(userId, p.authorId);
  res.json(store.update(id, fields(req.body, { title: 100, content: 5000 })));
});
```

동작과 응답 코드 설명

- **누구와 누구를 비교하나 :** 3.3에서 검증한 토큰의 사용자 id와, DB에서 읽은 글의 `authorId` 두 개를 비교합니다. 둘 다 숫자이고 `!==`로 비교합니다. 요청 본문이나 화면에서 온 값은 비교에 쓰지 않았습니다.
- **처리 순서 :** 수정은 다음과 같은 순서로 처리했습니다. 각 단계가 실패하면 그 단계의 오류가 나고, DB는 바뀌지 않도록 했습니다.

  1. 인증 (`currentUserId`): 401
  2. ID 형식 (`postId`): 400
  3. 글 존재 (`getPost`): 404
  4. 작성자 (`assertOwner`): 403
  5. 입력 (`fields`): 400
  6. 변경

  삭제의 경우 본문이 없으므로 5번의 입력 검사 없이, 작성자 확인 후 바로 삭제했습니다.

  그래서 같은 빈 제목을 23번 글(Alice 작성)에 보내도, Bob이 보내면 권한 단계에서 403 FORBIDDEN이고 작성자 Alice가 보내면 입력 단계에서 400 BAD_INPUT입니다 (T07_02). 토큰 없이 없는 글 999999를 고치려 하면 404가 아니라 401 TOKEN_REQUIRED입니다.

- **401과 403의 차이:**
  - 401은 "누구인지 확인할 수 없음"입니다. 토큰이 없거나, 틀렸거나, 만료된 경우입니다.
  - 403은 "누구인지는 알지만 이 글의 작성자가 아님"입니다. 로그인은 정상이므로 상태를 유지하고 "권한이 없습니다."만 표시합니다.
- **작성자 위조 방지 :** 등록 시 작성자는 `store.create(..., userId)`로 토큰에서 얻은 id만 사용합니다. `fields()`는 허용한 키(`title`, `content`) 외의 키가 하나라도 있으면 400 BAD_INPUT으로 거부합니다. 그래서 본문에 `authorId: 1`이나 `userId: 1`을 넣어도 글이 만들어지지 않습니다 (T12).
- **화면 버튼 :** `api.js` `canEdit()`이 로그인 사용자 id와 `post.authorId`를 비교해서, 다른 사람 글에는 수정·삭제 버튼을 보여 주지 않습니다. 이를 통해서 다른 작성자가 권한 없이 요청을 보내는 일을 프론트에서 방지했습니다. 하지만 이것은 프론트에서의 편의 기능이지 실제로 막는 것은 서버의 `assertOwner`이고, curl로 직접 보낸 요청도 403으로 거부하도록 했습니다.
- **성공 실패 응답 (HTTP와 Code)**
  - 작성자 본인의 요청이면 `assertOwner`는 통과시키고, 라우트가 성공 응답을 보냅니다: 등록 201 (+ `Location: /api/posts/{id}`), 수정 **200**, 삭제 **204** (본문 없음). 성공 응답에는 JSON `code`가 없습니다
- 토큰 사용자와 글의 `authorId`가 다르면 `assertOwner`가 403 FORBIDDEN을 던집니다.
- 본문에 `authorId`·`userId` 같은 허용되지 않은 필드가 있거나 제목·내용이 잘못되면 `fields()`가 400 BAD_INPUT을 던집니다.

- 인증에 실패하면 권한 검사 전에 `currentUserId`에서 401 (TOKEN_REQUIRED / INVALID_TOKEN / TOKEN_EXPIRED)로 끝나고, 없는 글이면 404 NOT_FOUND오류를 던집니다.

테스트 결과

실행 환경

- Windows + WSL2 Ubuntu 26.04.1 LTS / Node.js 24.21.0, npm 11.19.0, Python 3.14.4 / Chrome 155 / 백엔드 `npm start` (backend-express), 프론트 `npm run build` → `npm run preview` (frontend-vue)

로컬 배포 확인

- 새 폴더에 clone → 백엔드 `npm ci`, `.env.example` 복사 후 새 키 설정(예시 키 그대로면 시작 거부) → `npm start` / 프론트 `npm ci` → `npm run build` → `npm run preview` → 4173번 접속, 게시글 0개 목록으로 clone한 백엔드와의 API 연결 확인 (T00_02~T00_05, 2장 "로컬 실행 기록")

테스트 일시 및 코드 버전

2026-10-27 ~ 2026-10-28 (clone 재현은 10-28), 커밋 `55a0d93`과 같은 코드 (증거 촬영 후 코드 변경 없음)

토큰 TTL: 일반 테스트 90초 / 만료 테스트 `TOKEN_TTL=3 npm start`로 3초

ID	수행 절차와 사용자	예상 결과	실제 HTTP와 code	판정	증거
T01	비로그인으로 목록 조회 → 글 상세 조회	200	목록 200 / 해당 없음, 상세 200 / 해당 없음	PASS	T01_01_list, T01_02_detail
T02	Alice 로그인	200, JWT와 공개 사용자 정보	200 / 해당 없음	PASS	T02_01_login
T03	없는 이메일 (1234@class.test), Alice 틀린 비밀번호	401 INVALID_CREDENTIALS	없는 이메일 401 / INVALID_CREDENTIALS, 틀린 비밀번호 401 / INVALID_CREDENTIALS	PASS	T03_01_wrong_email, T03_02_wrong_pw, T00_01_check_final
T04	비로그인 화면에서 내 정보 조회, curl로 토큰 없이 글 등록	401 TOKEN_REQUIRED	/me 401 / TOKEN_REQUIRED, 등록 401 / TOKEN_REQUIRED	PASS	T04_01_me_no_token, T04_02_create_no_token, T00_01_check_final
T05	Alice 글 등록, 내 정보 조회	201 + Location, /me 200	등록 201 / 해당 없음, /me 200 / 해당 없음	PASS	T05_01_create, T05_02_me
T06	Alice 본인 글 수정, 본인 글 삭제	200, 204	수정 200 / 해당 없음, 삭제 204 / 해당 없음	PASS	T06_01_modify, T06_02_delete
T07	Bob 토큰으로 Alice 글 (23) 수정 시도, 전후 공개 조회. 추가로 같은 글에 빈 제목을 Bob과 Alice가 각각 전송	403 FORBIDDEN, 변경 없음	수정 403 / FORBIDDEN, 빈 제목 수정 Bob 403 / FORBIDDEN · Alice 400 / BAD_INPUT	PASS	T07_01_bob_modify, T07_02_bob_bad_input

ID	수행 절차와 사용자	예상 결과	실제 HTTP와 code	판정	증거
T08	Bob 토큰으로 Alice 글(23) 삭제 시도, 전후 공개 조회	403 FORBIDDEN, 변경 없음	삭제 403 / FORBIDDEN	PASS	T08_01_bob_delete
T09	Alice 토큰의 서명 첫 글자를 다른 문자로 바꿔 /me, POST, PUT, DELETE	401 INVALID_TOKEN, 변경 없음	/ me·POST·PUT·DELETE 모두 401 / INVALID_TOKEN	PASS	T09_01_command, T09_02_tamper_result1, T09_03_tamper_result2
T10	`TOKEN_TTL=3`으로 재시작, Alice 로그인 후 대기 → 내 정보 조회, 다시 로그인 후 대기 → 수정 저장	401 TOKEN_EXPIRED, 화면 로그인 해제	/me 401 / TOKEN_EXPIRED, 수정 401 / TOKEN_EXPIRED	PASS	T10_01_ttl3_start, T10_02_me_expired, T10_03_modify_expired
T11	Alice 로그인 → 로그아웃 → 내 정보 조회	401 TOKEN_REQUIRED	401 / TOKEN_REQUIRED	PASS	T11_01_logout_me
T12	Bob 토큰으로 본문에 `authorId:1`, `userId:1`을 각각 넣어 등록	400 BAD_INPUT, 위조 글 없음	authorId 400 / BAD_INPUT, userId 400 / BAD_INPUT	PASS	T12_01_forge_author
T13	Alice 토큰으로 빈 제목, 공백만 있는 본문, 101자 제목 등록	400 BAD_INPUT, DB 변경 없음	빈 제목·공백 본문·101자 제목 모두 400 / BAD_INPUT	PASS	T13_01_bad_input
T14	없는 ID 999999 조회·수정·삭제 (수정·삭제는 Alice 정상 토큰), 잘못된 ID `abc`·`0` 조회. 추가로 토큰 없이 999999 수정	없는 글 404, 잘못된 ID 400	없는 글 조회·수정·삭제 404 / NOT_FOUND, 잘못된 ID 400 / BAD_INPUT, 토큰 없이 수정 401 / TOKEN_REQUIRED	PASS	T14_01_command, T14_02_result1, T14_03_result2, T14_04_no_token_missing
T15	Alice로 "재시작 테스트" 글(24) 작성 → 백엔드 Ctrl+C → `npm start` → 조회	글·작성자·내용 유지	재시작 후 조회 200 / 해당 없음	PASS	T15_01_create, T15_02_server_stop, T15_04_server_restart, T15_03_after_restart
T16	Alice 로그인 → 새로고침	공개 글 유지, 로그인 해제	새로고침 후 목록 200 / 해당 없음	PASS	T16_01_before_refresh, T16_02_after_refresh
T17	토큰 없이 23번 수정·삭제, Bob 정상 작성 → 본인 수정 → 본인 삭제 → 조회	무토큰 401, Bob 본인 작업 성공	무토큰 수정·삭제 401 / TOKEN_REQUIRED, Bob 등록 201 / 해당 없음 → 수정 200 / 해당 없음 → 삭제 204 / 해당 없음 → 조회 404 / NOT_FOUND	PASS	T17_01_no_token, T17_02_bob_own

증거 파일은 모두 `docs/evidence/` 아래 PNG입니다. 토큰이 보이는 화면은 가렸고, curl 명령에서는 토큰을 셸 변수($A, $B, $BAD)에 담아 원문이 출력되지 않게 했습니다.

HTTP 응답 확인 기록

확인 항목	기대 응답	실제 HTTP와 code	증거 위치
정상 조회·로그인·수정	200	목록·상세 200, 로그인 200, `/me` 200, 본인 수정 200	T01, T02, T05_02, T06_01, T17_02
새 글 등록	201 및 Location	201 + `Location: /api/posts/22`, 201 + `Location: /api/posts/25`	T05_01, T17_02
본인 글 삭제	204 및 빈 본문	204 No Content, 본문 없음 (curl 출력에서 헤더 뒤 빈 줄만 있음)	T06_02, T17_02
입력 오류·추가 작성자 필드	400 BAD_INPUT	빈 제목·공백 본문·101자 400 / BAD_INPUT, `authorId`·`userId` 400 / BAD_INPUT, 잘못된 ID 400 / BAD_INPUT	T13, T12, T14
잘못된 계정	401 INVALID_CREDENTIALS	401 / INVALID_CREDENTIALS (화면 문구는 T03, JSON code는 자동 검사)	T03, T00_01
토큰 없음·헤더 형식 오류	401 TOKEN_REQUIRED	401 / TOKEN_REQUIRED (없는 글 수정 포함)	T04_02, T11, T17_01, T14_04
변조·클레임 오류	401 INVALID_TOKEN	401 / INVALID_TOKEN (서명 변조 /me·POST·PUT·DELETE)	T09
정상 서명의 만료 토큰	401 TOKEN_EXPIRED	401 / TOKEN_EXPIRED	T10_02, T10_03
다른 작성자의 변경	403 FORBIDDEN	403 / FORBIDDEN (수정·삭제, 잘못된 입력의 수정 포함)	T07, T08
없는 글	404 NOT_FOUND	404 / NOT_FOUND (조회·수정·삭제, 삭제 후 재조회)	T14, T17_02

WWW-Authenticate. 보호 API의 401 응답 모두에 `WWW-Authenticate: Bearer`가 있는 것을 확인했습니다. 확인한 화면은 T03, T04_01, T04_02, T09, T10_03, T11, T14_04, T17_01로 스크린샷을 찍었습니다. 이 헤더는 `app.js` 102행 오류 처리기가 상태가 401일 때 붙입니다.

401과 403의 화면 차이.

- 401이면 `request()`가 `clearSession()`을 호출합니다. 그래서 로그인 카드가 "로그아웃 상태"로 바뀌고, 로그아웃·등록 버튼이 비활성화되며, code별 재로그인 안내가 표시됩니다. (T10_02, T10_03).
- 403이면 세션을 지우지 않고, 로그인 상태를 유지한 채 "권한이 없습니다."만 표시합니다.
- 화면에서는 `canEdit()`이 다른 사람 글의 수정·삭제 버튼을 숨기기 때문에 403을 만들 수 없습니다. 그래서 403 응답 자체는 curl로 직접 요청을 보내 확인했습니다.

실패 후 데이터 불변.

실패 후 데이터가 불변인지, 실패를 통해 데이터가 바뀌지는 않았는지는 아래와 같은 항목들을 통해서 증명했습니다.

- T07, T08, T09, T17은 실패 요청 앞뒤로 같은 글을 공개 조회했고, 제목·내용·`updatedAt`이 같았습니다.
- T12, T13은 위조 글 개수(0개)와 전체 글 개수(전 4개 → 후 4개)로 새 글이 만들어지지 않았음을 확인했습니다.

토큰 없이 없는 글 수정 401 / 정상 토큰으로 없는 글 수정 404. 같은 999999번 수정 요청이 토큰에 따라 다르게 응답했습니다.

- 정상 Alice 토큰: 404 / NOT_FOUND (T14_02)
- 토큰 없음: 401 / TOKEN_REQUIRED + `WWW-Authenticate: Bearer` (T14_04)

수정 요청은 글을 찾기 전에 인증을 먼저 하기 때문에, 토큰 없는 수정 요청은 글 존재 확인까지 가지 않고 401로 끝납니다. 글이 있는지 자체는 공개 GET으로 누구나 조회할 수 있으므로, 이 순서는 존재를 숨기기 위한 것이 아니라 인증되지 않은 변경 요청을 가장 먼저 거르기 위한 것입니다.

권한과 입력 검사의 순서

23번 글(Alice 작성)에 같은 빈 제목 수정을 보냈습니다.

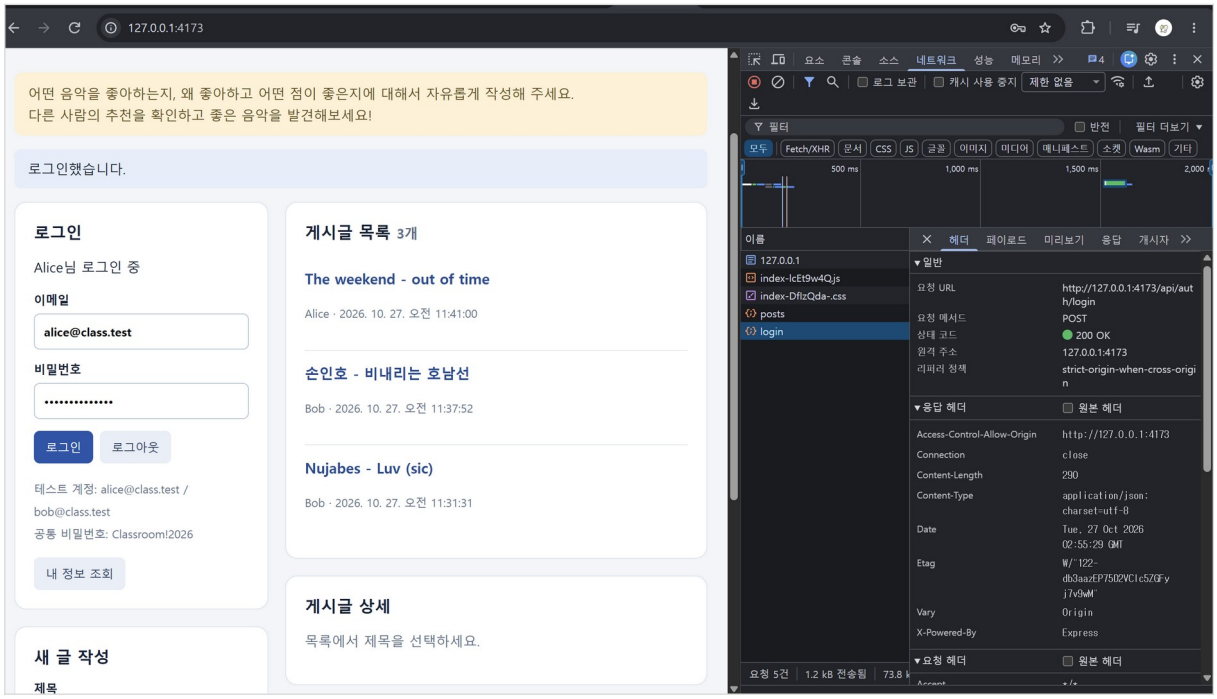
- Bob: 403 / FORBIDDEN
- Alice: 400 / BAD_INPUT

권한 검사(`assertOwner`)가 입력 검사(`fields`)보다 먼저라는 것을 확인했고, 두 요청 전후 23번 글은 같았습니다 .

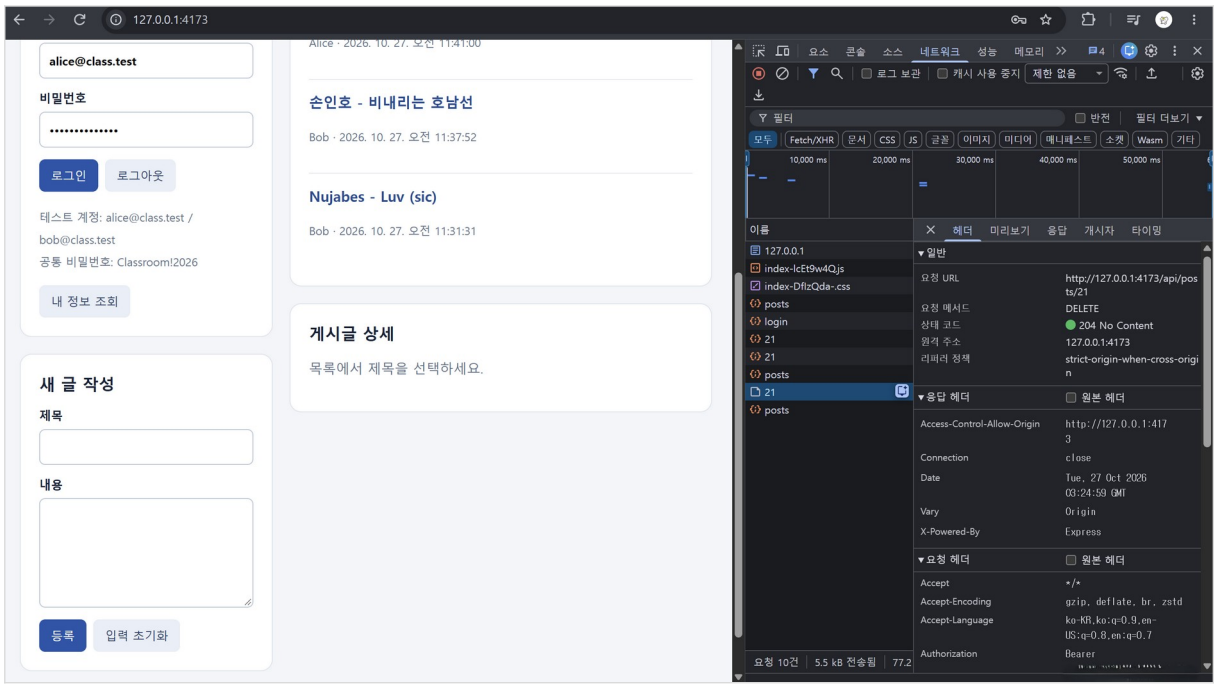
테스트 증거

로그인과 토큰 전송

증거 1: 로그인 성공과 토큰이 첨부된 요청. 로그인 200 후 "Alice님 로그인 중"이 표시되었습니다 (T02_01). 이어서 본인 글 삭제 요청의 요청 헤더에 `Authorization: Bearer <TOKEN>`이 붙어 있고, 204 No Content로 성공했습니다 (T06_02, 토큰 값은 가림). 등록 요청도 201 + `Location`으로 성공했습니다 (T05_01).



로그인 성공



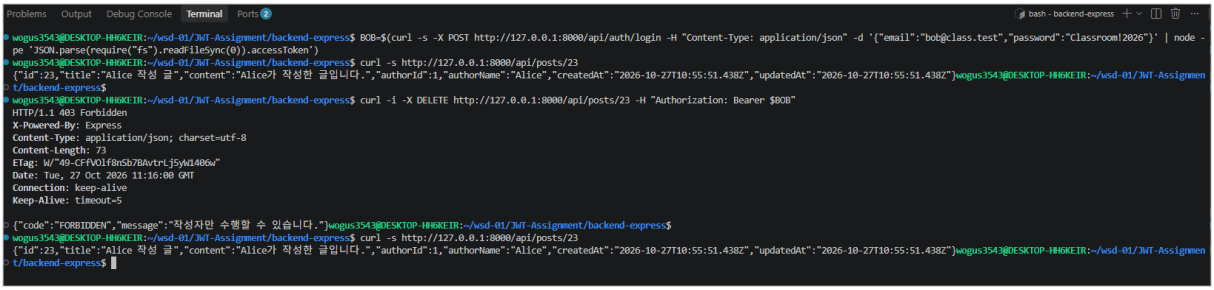
토큰이 첨부된 삭제 요청 204

다른 작성자의 수정과 삭제

증거 2: Bob이 Alice 글 수정·삭제를 시도한 서버 응답. 둘 다 403 FORBIDDEN이고, 전후 조회 결과가 같습니다.



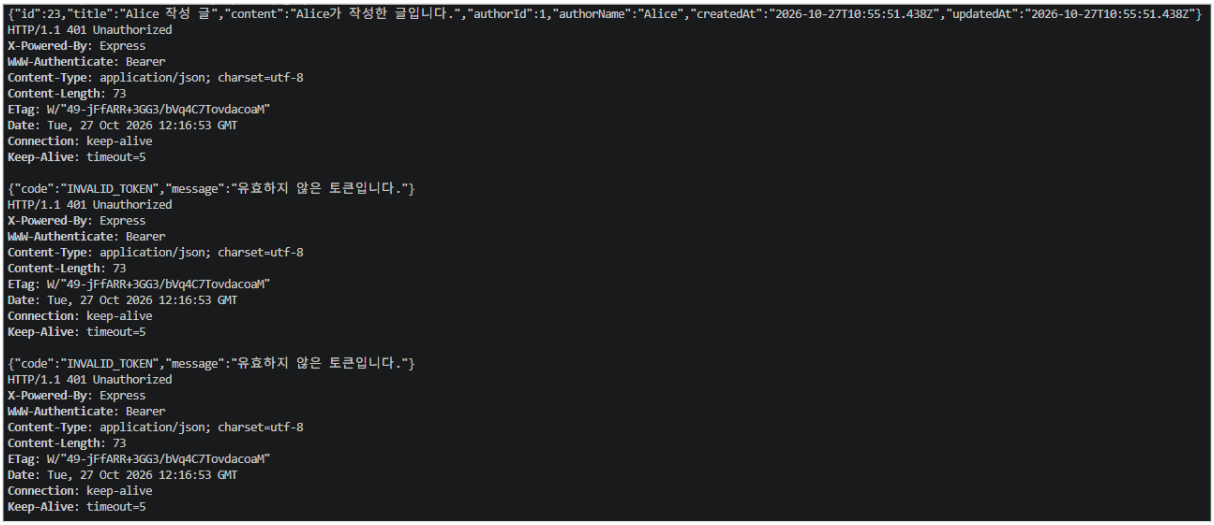
Bob 수정 403



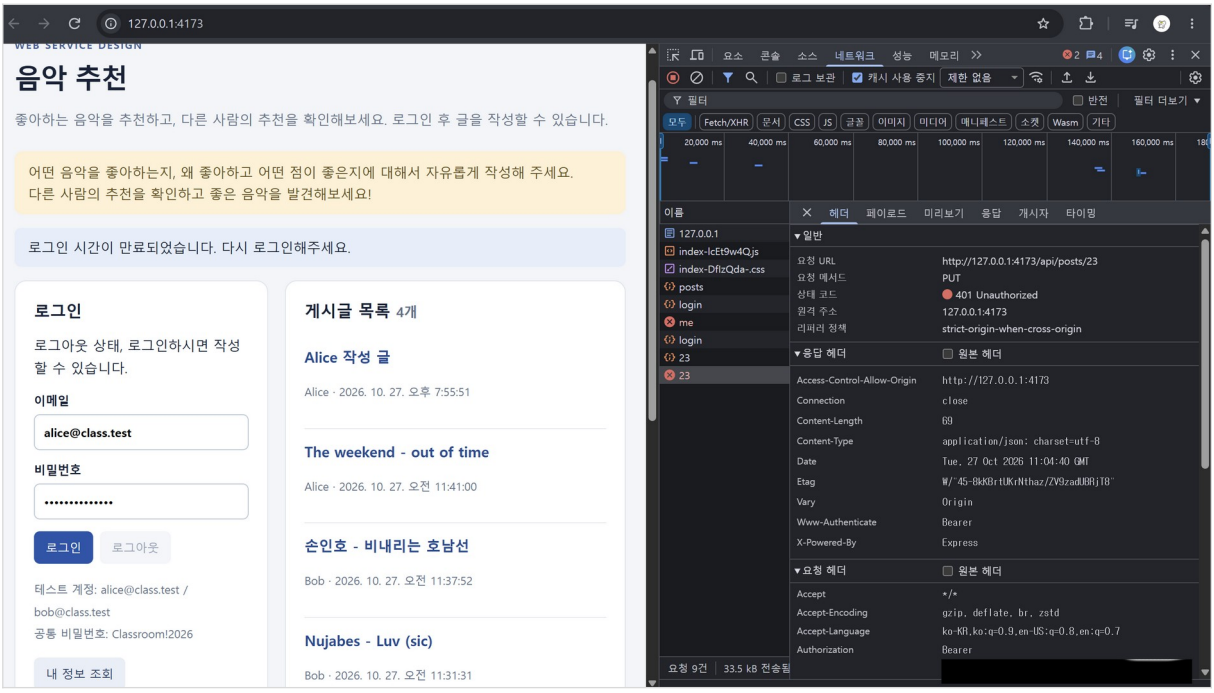
Bob 삭제 403

토큰 변조와 만료

증거 3: 토큰 변조·만료 시 서버 응답과 화면 처리. 서명 변조는 /me·POST·PUT·DELETE 모두 401 INVALID_TOKEN이었습니다. 만료는 401 TOKEN_EXPIRED와 함께 화면이 로그아웃 상태로 바뀌었습니다.



서명 변조 결과



만료 후 수정

자동 검사와 로컬 배포 증거

자동 검사.

- 백엔드를 켠 상태로 저장소 루트에서 `python3 tools/check_api.py --mode final`을 실행했고, **40개 검사 전체를 통과**했습니다 (`All 40 checks passed (final)`, T00_01_check_final). 이 검사는 잘못된 계정의 INVALID_CREDENTIALS, 토큰 없는 요청의 TOKEN_REQUIRED처럼 JSON code까지 확인합니다.
- 이 검사는 핵심 API 응답만 확인하고 화면 동작은 확인하지 않습니다. 그래서 위 T01~T17 수동 검사를 따로 수행했습니다.
- 만료 자동 검사(`--expiry`)는 실행하지 않았고, 대신 T10에서 수동으로 확인했습니다.

```
wogus3543@DESKTOP-HH6KEIR:~/wsd-01/JWT-Assignment$ python3 tools/check_api.py --mode final
PASS health
PASS public list
PASS bad ID
PASS missing ID
PASS wrong password
PASS unknown email
PASS no token me
PASS no token create
PASS login alice@class.test
PASS login bob@class.test
PASS Alice me
PASS Bob me
PASS create post
PASS public detail
PASS no token PUT
PASS Bob denied PUT
PASS failed request leaves data unchanged
PASS no token DELETE
PASS Bob denied DELETE
PASS failed request leaves data unchanged
PASS reject authorId
PASS reject userId
PASS input validation
PASS input validation
PASS input validation
PASS input validation
PASS tampered GET /me
PASS tampered POST /posts
PASS tampered PUT /posts/26
PASS tampered DELETE /posts/26
PASS failed request leaves data unchanged
PASS owner update
PASS create post
PASS Bob own update
PASS missing protected PUT
PASS missing protected DELETE
PASS delete owned test post
PASS deleted post missing
PASS delete owned test post
PASS deleted post missing
All 40 checks passed (final). UI and restart tests must be performed separately.
wogus3543@DESKTOP-HH6KEIR:~/wsd-01/JWT-Assignment$
```

T00_01 자동 final 검사 40개 전체 통과

```
wogus3543@DESKTOP-HH6KEIR:~/wsd-01$ git clone https://github.com/Jaehyeon3543/wsd-jwt.git wsd-jwt-check
Cloning into 'wsd-jwt-check'...
remote: Enumerating objects: 75, done.
remote: Counting objects: 100% (75/75), done.
remote: Compressing objects: 100% (71/71), done.
remote: Total 75 (delta 1), reused 75 (delta 1), pack-reused 0 (from 0)
Receiving objects: 100% (75/75), 5.50 MiB | 2.03 MiB/s, done.
Resolving deltas: 100% (1/1), done.
wogus3543@DESKTOP-HH6KEIR:~/wsd-01$ cd wsd-jwt-check/backend-express
wogus3543@DESKTOP-HH6KEIR:~/wsd-01/wsd-jwt-check/backend-express$ npm ci

added 81 packages, and audited 82 packages in 996ms

30 packages are looking for funding
  run `npm fund` for details

1 high severity vulnerability

To address all issues, run:
  npm audit fix

Run `npm audit` for details.
wogus3543@DESKTOP-HH6KEIR:~/wsd-01/wsd-jwt-check/backend-express$ cp .env.example .env
wogus3543@DESKTOP-HH6KEIR:~/wsd-01/wsd-jwt-check/backend-express$ npm start

> jwt-board-express@1.0.0 start
> node src/app.js

file:///home/wogus3543/wsd-01/wsd-jwt-check/backend-express/src/auth.js:10
  throw new Error("JWT_SECRET은 새로 생성한 key여야 합니다.");
        ^

Error: JWT_SECRET은 새로 생성한 key여야 합니다.
    at file:///home/wogus3543/wsd-01/wsd-jwt-check/backend-express/src/auth.js:10:9
    at ModuleJob.run (node:internal/modules/esm/module_job:561:25)
    at async node:internal/modules/esm/loader:647:26
    at async asyncRunEntryPointWithESMLoader (node:internal/modules/run_main:101:5)

Node.js v24.21.0
wogus3543@DESKTOP-HH6KEIR:~/wsd-01/wsd-jwt-check/backend-express$
```

clone 후 예시 키로 시작 거부

```
wogus3543@DESKTOP-HH6KEIR:~/wsd-01/wsd-jwt-check$ cd frontend-vue/
wogus3543@DESKTOP-HH6KEIR:~/wsd-01/wsd-jwt-check/frontend-vue$ npm ci

added 37 packages in 10s

10 packages are looking for funding
  run `npm fund` for details
wogus3543@DESKTOP-HH6KEIR:~/wsd-01/wsd-jwt-check/frontend-vue$ npm run build

> jwt-board-vue@1.0.0 build
> vite build

vite v8.3.4 building client environment for production...
✓ 12 modules transformed.
computing gzip size...
dist/index.html                  0.42 kB │ gzip:  0.31 kB
dist/assets/index-DfIzQda-.css   1.66 kB │ gzip:  0.80 kB
dist/assets/index-lcEt9w4Q.js   70.27 kB │ gzip: 27.81 kB

✓ built in 163ms
wogus3543@DESKTOP-HH6KEIR:~/wsd-01/wsd-jwt-check/frontend-vue$ npm run preview

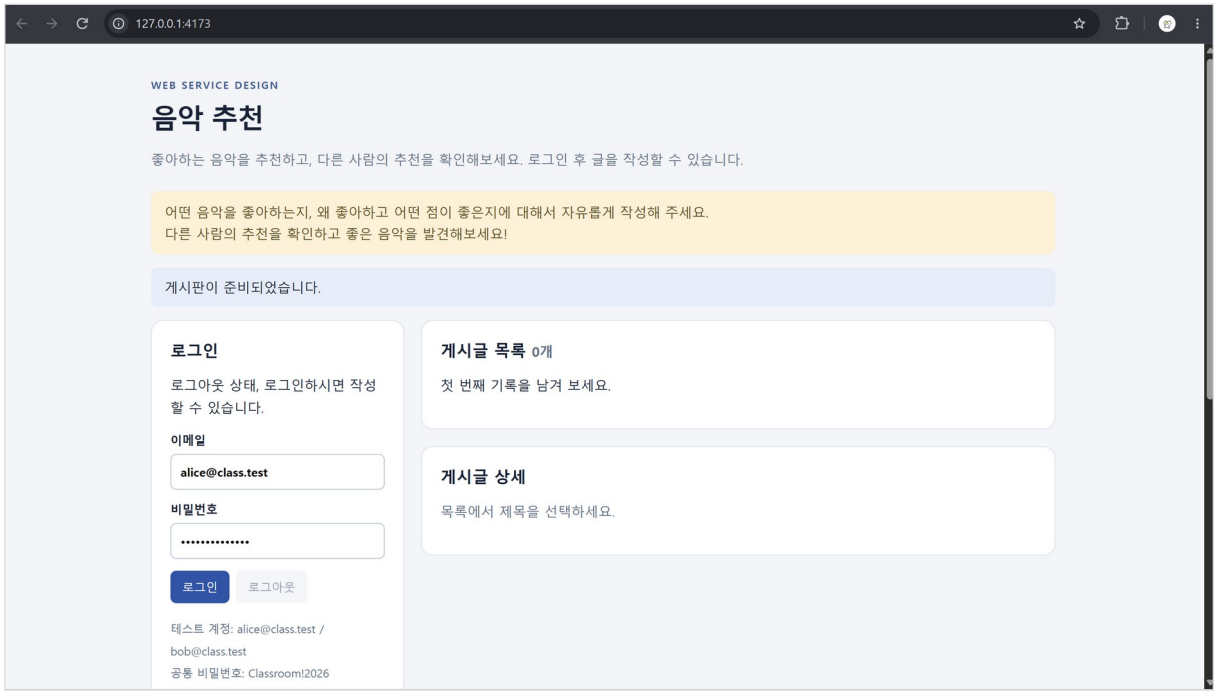
> jwt-board-vue@1.0.0 preview
> vite preview --host 127.0.0.1

error when starting preview server:
Error: Port 4173 is already in use
    at httpServerStart (file:///home/wogus3543/wsd-01/wsd-jwt-check/frontend-vue/node_modules/vite/dist/node/chunks/node.js:14706:10)
    at process.processTicksAndRejections (node:internal/process/task_queues:104:5)
    at async preview (file:///home/wogus3543/wsd-01/wsd-jwt-check/frontend-vue/node_modules/vite/dist/node/chunks/node.js:35726:2)
    at async CAC.<anonymous> (file:///home/wogus3543/wsd-01/wsd-jwt-check/frontend-vue/node_modules/vite/dist/node/cli.js:811:18)
wogus3543@DESKTOP-HH6KEIR:~/wsd-01/wsd-jwt-check/frontend-vue$ npm run preview

> jwt-board-vue@1.0.0 preview
> vite preview --host 127.0.0.1

  ➜ Local:   http://127.0.0.1:4173/
  ➜ press h + enter to show help
```

clone 폴더에서 빌드와 preview



clone 폴더의 4173번 화면



서버 재시작



재시작 후 재조회

AI 활용과 참고 자료

AI 사용 여부: 사용

구현 체크리스트를 한 단계씩 진행하면서 AI에게 개념 설명과 우선적으로 제가 고민해서 코드를 한번 작성해보고, 검토를 요청했습니다. 이후 AI의 제안에 따라 수정했습니다. `auth.js`, `api.js`, `App.vue`의 인증 코드는 직접 작성했고, AI가 찾은 오류의 원인을 듣고 직접 고친 뒤 요청을 보내 다시 확인했습니다. `auth.js`의 키 검사·32바이트 변환은 학습 과정에서 Codex가 제공한 예시를 참고했고, 최종 코드는 직접 작성·수정했습니다. 시작 코드의 입력 검사 결함 수정, 스크린샷 증거 점검, 테스트용 curl 명령, AI가 작성하거나 수행했습니다.

도구와 모델	사용 목적	반영한 파일 또는 보고서 항목	반영 방식
Claude Code (Claude Opus 5.5)	학습: JWT 구조, 서명 검증 원리, decode와 verify 차이, 401·403·404 구분, Vue ref와 일반 변수 차이	3장 설명 전체	참고
Claude Code (Claude Opus 5.5)	디버깅·리뷰: 작성한 코드의 오류 진단	`auth.js` `currentUserId`·`assertOwner`·키 검사, `api.js` `login`·`friendlyErrorMessage`·`canEdit`	수정 후 반영
Claude Code (Claude Opus 5.5)	생성: 시작 코드 결함 수정	`app.js` 14행·99~100행, `style.css` `button:disabled`	그대로 반영 (요청 후 직접 확인)
Claude Code (Claude Opus 5.5)	테스트: 토큰 21종 검증 스크립트, T07~T17 curl 명령, 스크린샷 검토	4장, `docs/evidence/`	수정 후 반영 (스크립트는 미제출)
Claude Code (Claude Opus 5.5)	제출 준비: `.env.example` 복원, Git 저장소 위치 점검	`.env.example`	그대로 반영
OpenAI Codex (GPT-6.1 sol High)	학습: 과제 요구사항, 프로젝트 구조, 요청 흐름 설명	2장 구조 이해	참고
OpenAI Codex (GPT-6.1 sol High)	학습: JWT 키 검사·32바이트 변환 예시 코드 제공	`auth.js` 최상단 키 검사 (5~15행)	참고 (최종 코드는 본인 작성·수정)

대표 사례

대표 사례 1: 서버 검증 함수의 오류 진단

도구 / 날짜: Claude Code (Claude Opus 5.5) / 2026-10

요청한 내용 : "함수를 작성했어. 이를 리뷰해주고, 어느 부분이 어떤 것을 검증하는지 설명해. 테스트 방법도 제공해." 그다음 "테스트 실패가 몇개 존재해. 원인을 찾고 나에게 보고해"

AI가 제안한 내용: 처음 작성한 `currentUserId`에서 다음 문제들을 찾았습니다.

- `catch (error instanceof ...)`는 문법 오류였습니다.
- `try` 블록 안에서 `const`로 선언한 변수를 블록 밖에서 사용하고 있었고, 그 이름(p)과 실제로 쓰는 이름(`payload`)도 서로 달랐습니다.
- `findPublic`을 `findPulic`으로 잘못 쓴 오타가 두 단계로 문제를 일으켰습니다.
    - 처음에는 import 줄과 호출 줄 둘 다 오타였습니다. ESM은 서버 시작 때 import를 연결하므로, `users.js`에 없는 이름을 가져오려다 `SyntaxError: The requested module './users.js' does not provide an export named 'findPulic'`로 서버가 아예 시작되지 않았습니다. `node --check`는 문법만 보기 때문에 이 오류를 잡지 못했습니다.
    - import 줄만 고친 뒤에는 서버가 시작됐습니다. 하지만 함수 본문의 호출 줄은 그대로 오타라서, 21종 검사 중 서명·클레임 검사를 모두 통과해 그 줄까지 도달한 3개(정상 Alice, 정상 Bob, `sub=999`)만 500 INTERNAL_ERROR가 났습니다. 나머지 18개는 그 줄 전에 401로 끝났습니다.
- 함수가 사용자 id가 아니라 사용자 객체를 반환하고 있다는 것도 알려 줬습니다.
- 확인용으로 잘못된 토큰 21종을 만들어 `/api/me`에 보내는 스크립트를 제공했습니다.

실제 반영 위치

- `backend-express/src/auth.js` `currentUserId` (44~72행)

제가 확인하고 수정한 내용: 다음과 같이 직접 고쳤습니다.

- `let payload`를 `try` 밖에 선언하고, `catch (error)` 안에서 `if (error instanceof jwt.TokenExpiredError)`로 분기했습니다.
- `findPulic` 오타를 import 줄부터 고쳤고, 서버가 켜진 뒤 검사에서 3개가 500인 것을 보고 호출 줄도 고쳤습니다.
- `return user.id`로 바꿨습니다. 라우트가 이 값을 `assertOwner`에서 글의 `authorId`(숫자)와 `!==`로 비교하기 때문에, 객체를 반환하면 본인 글도 항상 403이 되기 때문입니다.

비슷한 실수로 `throw ApiError(...)`에 `new`가 빠져 500이 나던 것도 같은 방식으로 고쳤습니다.

검증 방법과 결과:

- 수정 전에는 정상 Alice 토큰이 500이었습니다.
- 수정 후 21종 스크립트가 21/21 기대값과 일치했습니다. 정상 Alice·Bob은 200, 만료는 TOKEN_EXPIRED, 나머지는 TOKEN_REQUIRED 또는 INVALID_TOKEN이었습니다.
- 이후 curl로 Bob의 토큰을 사용해 Alice 글을 수정하려 할 때 403이 나는 것도 확인했습니다 (T07).

대표 사례 2: 프론트 401 처리 위치

도구 / 날짜: Claude Code (Claude Opus 5.5) / 2026-10

요청한 내용: "직관적으로 보기 편하게 if문으로 작성해봤어. 리뷰해줘." (401 code별 안내 문구를 작성한 뒤)

AI가 제안한 내용: if 문으로 작성한 방식은 유지했고, 오류를 고쳤습니다.

- if 문으로 쓰고 싶었던 방식은 유지했습니다.
- 위치만 `friendlyErrorMessage()`의 `status === 401` 블록으로 옮기고, 비교 대상을 `data.code`로 바꿨습니다.
- `${data.message}`를 큰따옴표 문자열 안에 써서 변수가 들어가지 않던 것을 백틱으로 고쳤습니다.)

실제 반영 위치: `frontend-vue/src/api.js` `friendlyErrorMessage` (18~32행), `request` (52~60행)

검증 방법과 결과: 각 code를 실제로 만들어 화면 문구를 확인했습니다.

- INVALID_CREDENTIALS: 틀린 비밀번호 (T03)
- TOKEN_REQUIRED: 로그아웃 후 내 정보 조회 (T11)
- TOKEN_EXPIRED: `TOKEN_TTL=3` (T10)
- INVALID_TOKEN: 로그인한 상태에서 서버의 `JWT_SECRET`을 바꿔 재시작한 뒤 요청

네 경우 모두 해당 문구가 나왔습니다. 로그인 실패(INVALID_CREDENTIALS)는 원래 로그아웃 상태 그대로였고, 나머지 세 경우는 로그인 상태에서 로그아웃 상태로 바뀌었습니다.

대표 사례 3: 시작 코드의 입력 검사 결함

도구 / 날짜: Claude Code (Claude Opus 5.5) / 2026-10

요청한 내용: 입력 검사 체크리스트를 확인하던 중 AI가 결함을 보고했고, 저는 "결함에 대해 설명해줘. 확인하고 내 능력 밖이면 너가 수정해줘"라고 요청했습니다.

AI가 제안한 내용: 시작 코드 `app.js`에서 두 가지 결함을 찾았습니다.

- `fields()`의 `!(k in limits)` 검사는 `in`이 프로토타입 체인까지 보기 때문에, 본문에 `toString`이나 `constructor` 키를 넣어도 201로 글이 만들어졌습니다.
- 64KB를 넘는 본문은 express.json이 던진 오류가 처리되지 않아 500 INTERNAL_ERROR로 응답했습니다.

AI는 각각 `!Object.hasOwn(limits, k)`와, body-parser가 클라이언트 오류에 붙이는 `error.expose`가 참이면 400으로 바꿨습니다.

실제 반영 위치

- `backend-express/src/app.js` 14행, 99~100행

제가 확인하고 수정한 내용: 코드는 AI 수정 그대로 반영했습니다. `in`과 `Object.hasOwn`의 차이(상속된 키 포함 여부)에 대한 설명을 듣고, 추가 필드를 막는 목적에는 객체 자신의 키만 봐야 한다는 점을 이해했습니다.

검증 방법과 결과:

- AI가 실제 DB를 건드리지 않도록 별도 포트(8001)와 임시 DB로 서버를 띄워 확인했습니다. 70KB 본문은 400, `toString`·`constructor`·`__proto__` 키는 400이었고, 정상 등록·수정·삭제는 그대로 동작했습니다.
- 저는 8000번 서버를 재시작한 뒤 `check_api.py --mode final` 통과와 T12·T13 결과로 다시 확인했습니다.

참고 자료

자료 제목과 URL	사용한 위치	반영 내용 및 출처 표시
웹서비스설계 수업 자료와 제공 시작 코드 (`docs/01~07`, `backend-express`, `frontend-vue`, `tools/check_api.py`)	전체 구조, `store.js`, `users.js`, 라우트, 검사 스크립트	제공 코드 유지, 인증 TODO 구현
node-jsonwebtoken README, https://github.com/auth0/node-jsonwebtoken	`auth.js` `issueToken`, `currentUserId`	AI 설명의 근거 자료 (직접 열람하지 않음): `jwt.sign`·`jwt.verify` 옵션(`algorithms`, `issuer`, `audience`, `subject`, `expiresIn`)과 `TokenExpiredError`
RFC 7519 JSON Web Token, https://www.rfc-editor.org/rfc/rfc7519	3.1, 3.3 설명	AI 설명의 근거 자료 (직접 열람하지 않음): `sub`·`iss`·`aud`·`exp` 클레임 의미

자료 제목과 URL	사용한 위치	반영 내용 및 출처 표시
MDN `Object.hasOwn()`, https://developer.mozilla.org/docs/Web/JavaScript/Reference/Global_Objects/Object/hasOwn	`app.js` `fields`	AI 설명의 근거 자료 (직접 열람하지 않음): `in` 연산자와의 차이

직접 참고한 자료는 수업 자료와 제공 시작 코드입니다. 외부 자료 3개는 AI가 설명할 때 근거로 삼은 자료이며, 직접 열어 보지는 않았습니다.

이해도 확인과 회고

1. 요청 본문의 authorId를 믿으면 어떤 문제가 생기며, 내 코드는 어떻게 막는가?

- 요청 본문의 authorId를 믿으면 다른 사용자가 자신의 토큰을 이용한 채 다른 사용자의 id (authorId)을 보내 타인의 id로 글을 등록·수정·삭제가 가능해집니다.

글의 등록의 경우 app.js에서 등록 라우터에 작성자 currentUserID (서명이 검증된 토큰) 의 sub에서만 가져와서 store.create(..., userId)에 넘깁니다. 그렇기에 작성자의 id를 본문에서 가져오는게 아니라 토큰에서 가져오기에 안전합니다. 그 외의 수정과 삭제의 경우 토큰에서 검증한 id와 DB에서의 authorId가 같은지 검사하는 asserOwner를 통해서도 막을 수 있습니다. 또한 field( ) 를 통해서도 막을 수 있습니다. 지정한 필드가 아닌 다른 필드가 들어오면 400 BAD_INPUT 오류를 던집니다.

2. 수정 버튼을 숨기는 것만으로 충분하지 않은 이유는 무엇이며, 서버의 어떤 함수가 권한을 검사하는가?

버튼은 단순히 브라우저의 화면일 뿐이고, 서버 API는 화면을 거치지 않고도 직접 호출할 수 있습니다. test에 사용한 curl이나 개발자 도구로 요청을 보내면 버튼이 없어도 실제로 요청이 서버에 도착할 수 있습니다. 그렇기에 실제 검사는 서버의 auth.js의 asserOwner( ) 가 합니다. 이 함수가 수정 그리고 삭제 라우트에서 글을 DB에서 읽은 다음에 토큰의 사용자 id와 글의 authorId가 다르다면 403 FORBIDDEN 오류를 던집니다. 이에 대해서는 실제로 T07, T08에서 화면 버튼에서 요청을 보낸게 아닌 curl 로 Alice 글의 수정, 삭제 요청을 보냈을 때 403 오류를 던지는걸 볼 수 있습니다.

3. JWT decode와 verify의 차이는 무엇이며, 제 코드는 어떤 검증을 수행하는가?

JWT의 헤더와 페이로드는 암호화가 아닙니다. Base64URL 인코딩이기 때문에 누구나 풀어서 읽을 수 있습니다. 그렇기에 decode는 내용을 풀어서 꺼내기만 하고 실제로 그 내용이 서버가 만든 것인지는 확인하지 않습니다. 반면 verify는 헤더와 페이로드를 서버 키로 다시 HMAC 계산해서 토큰의 서명과 같은지를 비교합니다. 그래서 누군가가 sub를 바꾼다면 계산이 달라지기 때문에 실패합니다. 제 코드는 currentUserId를 통해서 다음의 단계를 차례대로 검사합니다.

1. jwt.verify로 서명을 검증합니다. 이때 알고리즘을 고정하고 iss, aud 값과 exp 만료도 함께 검사합니다.

2. verify검사에서 마저 하지 못한 exp가 숫자로 존재하는지, sub가 양의 정수 문자열인지도 직접 확인합니다.

3. 마지막으로 sub에 해당하는 사용자가 실제로 DB에 존재하는지도 확인합니다.

4. 로그아웃, 토큰 만료, 새로고침은 각각 무엇을 바꾸는가? 이미 복사된 토큰은 어떻게 되는가?

- **로그아웃 -** 브라우저 메모리의 `session`을 비우고 화면 상태를 바꿉니다. 토큰과 사용자 정보를 삭제하고 화면용 로그인 상태를 지웁니다. 그리고 작성 및 수정중인 입력도 초기화합니다.
- **토큰 만료:** 서버의 판단이 바뀝니다. `exp`가 지나면 같은 토큰이 401 TOKEN_EXPIRED가 되고, 프론트는 그 401을 받은 순간 세션을 지웁니다
- **새로고침:** JS가 다시 실행되면서 `session` 변수가 초기값이 되어 로그인만 풀립니다. 글은 서버 DB에 있으므로 그대로입니다

이미 복사된 토큰은 로그아웃을 해도 무효가 되지 않습니다. logout은 단순히 브라우저 메모리의 session을 비운것 뿐이니 복사된 토큰이 있다면 exp 시간 안에 사용이 가능합니다. 그렇기에 토큰 자체를 훔치기 어렵게 만들어야 합니다.

5. 가장 어려웠던 오류 한 가지와 원인 확인·수정·재검증 과정을 설명하라.

프론트에서Alice로 로그인해서 Alice 글을 수정했는데 "토큰이 없다"는 401 TOKEN_REQUIRED가 나온 일입니다. 원인은 FE-02 `authorizationHeaders()`가 아직 빈 객체를 반환해 헤더가 붙지 않은 것이었습니다. 토큰은 발급받았지만 아직 요청에 토큰을 포함시키지 않았습니다. Network 탭 요청 헤더에 Authorization이 없는 것과 curl에 토큰을 직접 넣으면 성공하는 것으로 확인했습니다. 흔하게 일어날 수 있는 에러였으며 이를 통해 오류의 상태코드와 메세지를 통해 어떤 점에서 오류가 났는지 빠른 인지가 가능했고 빠른 대처가 가능했습니다.

FE-02를 구현해 요청에 토큰을 반환하도록 구현했고, 본인 글 수정과 삭제 등의 요청 헤더에 토큰이 붙는 것을 확인했습니다.

이 과정에서 오류 메세지와 상태 코드가 중요하다는 사실을 알게 되었습니다. 오류가 어떤 부분에서 났는지 알려주니 다른 부분을 다 열어보지 않고도 오류가 일어난 부분만 확인해서 수정하면 된다는 사실이 정말 다행이라 생각했습니다.

현재 남은 문제와 다음에 개선할 점:

- 새로고침하면 로그인이 풀립니다. 보안상 의도한 설계이지만 사용자에게는 불편합니다. 개선한다면 HttpOnly 쿠키의 refresh token처럼, JS가 읽을 수 없는 저장 방식을 찾아보겠습니다.
- 로그아웃이 서버 토큰을 폐기하지 않습니다. 만약 토큰이 복사된다면 다른 사람이 악용할 가능성이 있습니다. 이는 토큰을 확실하게 숨기거나 혹은 로그아웃 시 서버에서 토큰을 폐기하는 방향으로 개선해야할거 같습니다.